

A Custom Autonomy Stack for a Warehouse AMR

Juan José Jáuregui Barba, Victor Alejandro Meneses Garza, Rosendo De Los Rios Moreno,
Jordan Palafox Salinas, and Hugo Daniel Castillo Ovando
Tecnológico de Monterrey — TE3003B, FJ2026
{A00836722, A01384002, A01198515, A00835705, A00836025}@tec.mx

Abstract—This report documents the complete final system of the TE3003B *Puzzlebot Challenge* project: an autonomous forklift AMR that executes full pick-and-place warehouse missions on a 3.65×4.85 m scaled track. The stack is entirely custom-built — no Nav2, no Whisper, no `robot_localization`. The core subsystems are: (1) a C++ MCL/AMCL SLAM node with optional CUDA scoring, ICP scan-to-scan and scan-to-map refinement, and a graph-SLAM back-end for loop closure; (2) an ArUco beacon localization layer (`aruco_localization`) that rescues the particle filter when it drifts in low-feature corridors; (3) A* + Bug1 navigation with pure-pursuit trajectory tracking; (4) a YASMIN state machine that orchestrates full pick-and-place missions across three zone types (rollers, racks, trucks); (5) a YOLOv8 logo classifier and QR-based docking for visual alignment at pick/place sites; (6) an MFCC + VQ + HMM voice pipeline for command dispatch; (7) a Flask + SocketIO web dashboard with live telemetry and camera streaming; (8) a Tang Nano 20K FPGA lifter controlled via SPI from the Jetson Nano; and (9) a live Gazebo digital twin that mirrors the physical robot in real time. Two full pick-and-place missions were validated end-to-end on hardware: Mission 1 (Rollers → Truck) and Mission 2 (Racks → Truck).

Index Terms—ROS 2, Puzzlebot, MCL, AMCL, CUDA, ICP, graph SLAM, loop closure, ArUco, localization, A*, Bug1, pure-pursuit, YASMIN, FPGA, SPI, YOLOv8, logo classifier, HMM, voice, Flask, SocketIO, digital twin, Gazebo

I. INTRODUCTION

The *Puzzlebot Challenge* (TE3003B, ITESM FJ2026) asks teams to build a fully autonomous forklift AMR that picks pallets from one zone and delivers them to another inside a scaled warehouse. The arena contains three zone types: **Trucks** (loading docks, 3 units), **Racks** (two-level shelving), and **Rollers** (roller-table staging areas).

The team’s principal constraint, maintained from Sprint 1 to the final demonstration, was to implement every component from scratch: **Nav2 is not used, Whisper is not used, `robot_localization` is not used**. The system is distributed across two compute nodes: a **Jetson Nano 4 GB** on-board (sensing, SLAM, lifting) and a **laptop** off-board (navigation, mission control, dashboard, voice, digital twin), connected over Wi-Fi through shared DDS with `ROS_DOMAIN_ID`.

Figure 1 shows the physical competition track and the Puzzlebot.

A. Objectives and Contributions

The project pursued one functional goal — complete a full *pick-and-place* mission autonomously — decomposed into nine engineering objectives, one per subsystem (localization,

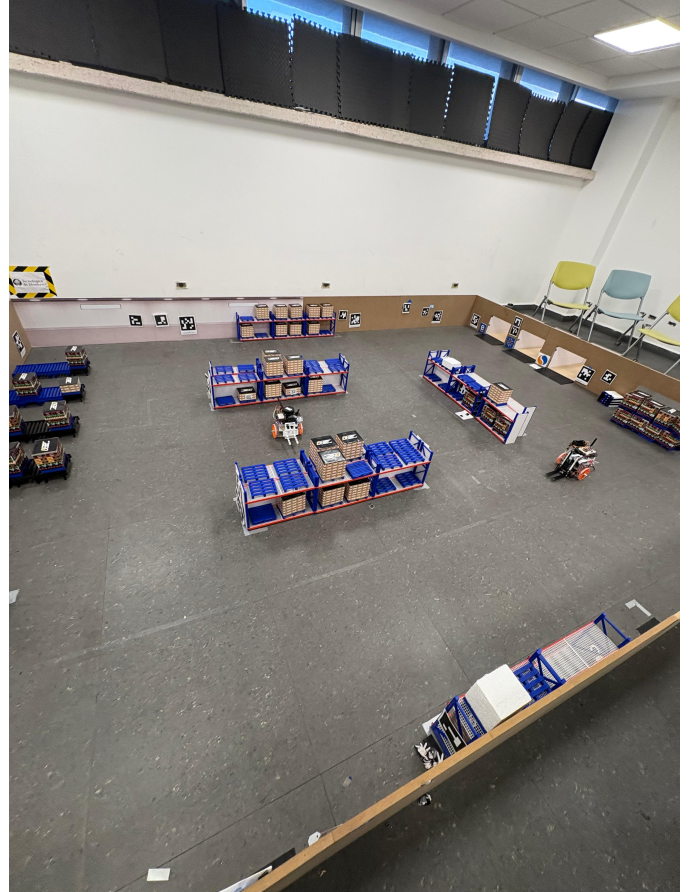


Figure 1. The 3.65×4.85 m competition track with the Puzzlebot AMR. Trucks are at the far end; racks (two levels) occupy the centre; roller tables are at the near end.

navigation, mission orchestration, lifting, perception, voice, dashboard, digital twin, and safety). Relative to a baseline that would have stitched together off-the-shelf ROS 2 packages, the principal contributions of this work are:

- 1) A **from-scratch C++ Monte-Carlo localization** node with an optional CUDA likelihood kernel, ICP refinement, and a graph-SLAM back-end for loop closure — a drop-in replacement for `slam_toolbox` + `amcl`.
- 2) An **ArUco beacon localization layer** that acts as a rescue seed for the particle filter, eliminating the orientation drift that featureless corridors induce in pure laser MCL.
- 3) A **self-contained navigation stack** (A*, Bug1, pure-pursuit, local costmap) that replaces Nav2 while remaining

small enough to read and tune by hand.

- 4) A **deterministic mission orchestrator** built on YASMIN with a live JSON debugger for field bring-up.
- 5) A **custom voice front-end** (MFCC + VQ + HMM) that runs on CPU with no pre-trained network and no cloud dependency.

B. Report Organization

Section II describes the distributed deployment and the ROS 2 graph. Section III details each subsystem — SLAM, navigation, mission control, lifting, perception, voice, dashboard, digital twin, and the industrial-standards mapping. Section IV collects the integration problems that were diagnosed and fixed, Section V reports the end-to-end mission results, and Section ?? gives the final task backlog.

II. FINAL SYSTEM ARCHITECTURE

A. Distributed Deployment

The system runs as two launch groups on the same ROS_DOMAIN_ID:

- **Jetson Nano** (robot.launch.py): LiDAR driver, real_odom (differential kinematics + TF), slam_node (C++ MCL), aruco_localization, lifting_node (SPI → FPGA), qr_quad_alignment, logo_stop_debug, twist_relay.
- **Laptop** (laptop.launch.py): nav_node (A* + Bug1 + pure-pursuit), state_machine_node (YASMIN), dashboard_node (Flask), voice_node (HMM), RViz 2, Gazebo digital twin.

a) *Anti-brownout relay*: all upstream nodes publish velocity commands to /cmd_vel_in; the twist_relay node applies a ramp filter and forwards to /cmd_vel → micro-ROS → motors. This prevents voltage-induced Jetson resets from sudden current spikes.

Table I
ROS 2 PACKAGES IN THE FINAL SYSTEM

Package	Lang	Purpose
slam	C++	MCL/AMCL + CUDA + graph SLAM
localization	C++	EKF + ICP velocity bridge
controller	Python	Kinematics, odometry, twist relay
navigation	Python	A* + Bug1 + pure-pursuit
mission_control	Python	YASMIN state machine
perception	Python	ArUco, QR dock, logo YOLO
lifting	Python	HAL SPI → FPGA
voice_control	Python	MFCC + VQ + HMM
dashboard	Python	Flask + SocketIO
description	CMake	URDF, Gazebo world, digital twin
bringup	CMake	Launch files

B. Coordinate Frames and the TF Tree

All geometry is expressed in a standard REP-105 frame chain. The map frame is the static world anchored to the saved occupancy grid; odom is a smooth, drift-prone frame produced

by wheel odometry; base_link is the robot body; and laser and camera_link are the sensor frames fixed by the URDF. The localization stack publishes the map → odom correction, while real_odom publishes odom → base_link:

$$T_{\text{map}}^{\text{base}} = \underbrace{T_{\text{map}}^{\text{odom}}}_{\text{SLAM/MCL}} \cdot \underbrace{T_{\text{odom}}^{\text{base}}}_{\text{wheel odometry}}.$$

Decoupling the two transforms this way is what lets MCL apply a discrete pose correction (which would otherwise teleport the robot) without ever introducing a discontinuity into the high-rate odom frame that the controller integrates.

C. Data Flow

Sensor topics (/scan, /cam_img, the encoder velocities) originate on the Jetson and are consumed locally by SLAM and perception; only compact estimates (/slam_pose, /map, /aruco_pose_estimate, /logo_detection) cross the Wi-Fi link to the laptop, keeping the wireless DDS traffic well below the bandwidth that raw images would demand. Command topics flow the other way: the planner and the state machine publish to /cmd_vel_in, the relay ramps them, and /cmd_vel is bridged to the motors through micro-ROS. Figure 2 shows the full graph.

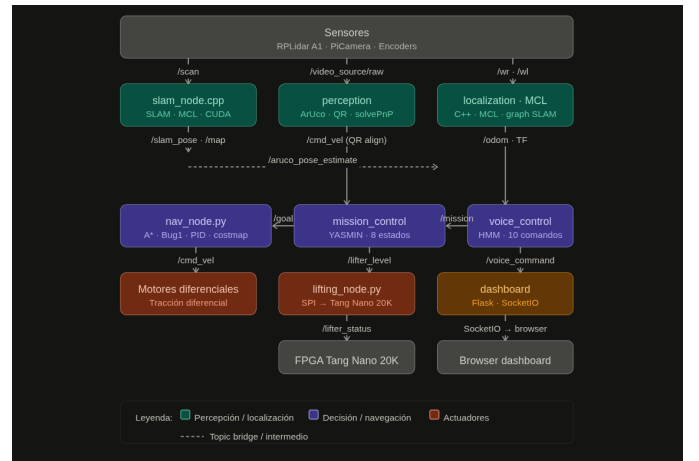


Figure 2. End-to-end ROS 2 data flow. On-board nodes (Jetson) handle sensing, SLAM, perception, and lifting; off-board nodes (laptop) handle planning, mission control, the dashboard, and the digital twin. Only compact estimates cross the Wi-Fi/DDS boundary.

III. SUBSYSTEM DESCRIPTIONS

A. M3. SLAM and Localization

a) *C++ MCL/AMCL node*: slam_node (C++) runs the full particle-filter loop at 10 Hz on every /scan arrival:

- 1) **Predict** — wheel odometry + ICP scan-to-scan as a slip-robust rotation prior.
- 2) **Update** — likelihood-field sensor model scores each particle against the wall map; a CUDA kernel accelerates this step when the GPU is available.

- 3) **Resample** — low-variance resampling with recovery (random injection) triggered when n_{eff} falls below 15% of 600 particles.
- 4) **Scan-to-map refine** — ICP over a 4 m search radius, 0.3 m maximum correction, with a 15° coarse yaw sweep.

A back-end thread runs **graph SLAM**: it accumulates keyframes and detects loop closures, then re-rasterizes the map. Two operating modes are provided: mapping (build a new .pgm) and navigation (load the saved map, MCL only).

Map parameters: 0.04 m cells, 400×400 grid (16×16 m), incremental log-odds (occupied $+0.5$, free -0.2).

b) Motion model: Each particle i is propagated with the odometry sample model: the wheel displacement $(\delta_{\text{trans}}, \delta_{\text{rot1}}, \delta_{\text{rot2}})$ is perturbed by zero-mean Gaussian noise whose variance scales with the motion magnitude,

$$\hat{\delta}_{\text{rot}} = \delta_{\text{rot}} - \mathcal{N}(0, \alpha_1 \delta_{\text{rot}}^2 + \alpha_2 \delta_{\text{trans}}^2),$$

so that fast or sharply-turning motion inflates the cloud and slow straight motion keeps it tight. The ICP scan-to-scan estimate replaces the raw wheel rotation whenever the two disagree by more than a threshold, which makes the prior robust to wheel slip on the track's smooth surface.

c) Sensor model: The likelihood field pre-computes, for every free cell, the Euclidean distance d to the nearest occupied cell. A beam endpoint falling on a cell of distance d contributes

$$p(z | x) \propto z_{\text{hit}} e^{-d^2/2\sigma^2} + z_{\text{rand}},$$

and the per-particle weight is the product over a subsampled set of beams (every k -th ray) for speed. The CUDA kernel evaluates all 600 particles against all beams in parallel; on the Jetson's GPU this cut the update step from the dominant cost of the loop to a negligible fraction, which is what allowed the full filter to hold 10 Hz.

d) ArUco rescue beacons: `aruco_localization` (Python, perception package) operates 20 fixed floor markers (dictionary `DICT_ARUCO_ORIGINAL`, 9 cm side). For each detection it computes the robot's absolute pose $T_{\text{map}}^{\text{base}} = T_{\text{map}}^{\text{marker}} \cdot (T_{\text{cam}}^{\text{marker}})^{-1} \cdot (T_{\text{base}}^{\text{cam}})^{-1}$ and publishes to `/aruco_pose_estimate` (`PoseWithCovarianceStamped`). When two or more markers are visible simultaneously, a **2-D Umeyama rigid registration** fits the observed positions to the canonical map, rejecting outliers greedily; baseline ≥ 0.25 m and fit RMSE ≤ 0.10 m are required to publish. `slam_node` consumes the estimate as a rescue seed: if the MCL belief disagrees by more than `aruco_snap_tol`, the particle cloud is re-centred there and the laser snaps it to the walls.

A visual web editor (`perception/tools/aruco_map_editor`) allows the map to be updated interactively at <http://127.0.0.1:8770>.

B. MR. Navigation

`nav_node` loads the pre-saved map (.pgm + .yaml) and `waypoints.yaml` at startup. Named zones follow the convention `truck_*`, `rack_*`, `roller_*`.

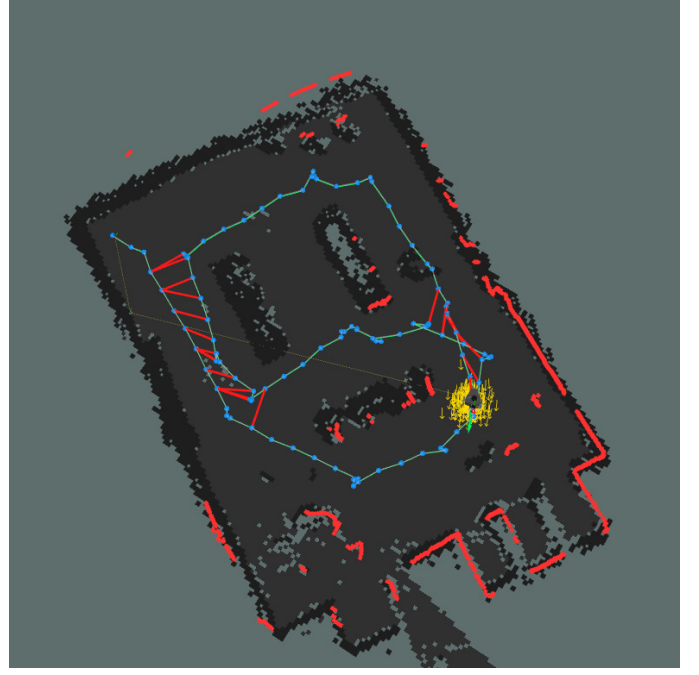


Figure 3. RViz 2 view of the final localization stack: occupancy grid, MCL particle cloud (arrows), RPLidar A1 scan overlay, and the ArUco detection debug image (inset top-right) showing detected marker IDs and their estimated poses.

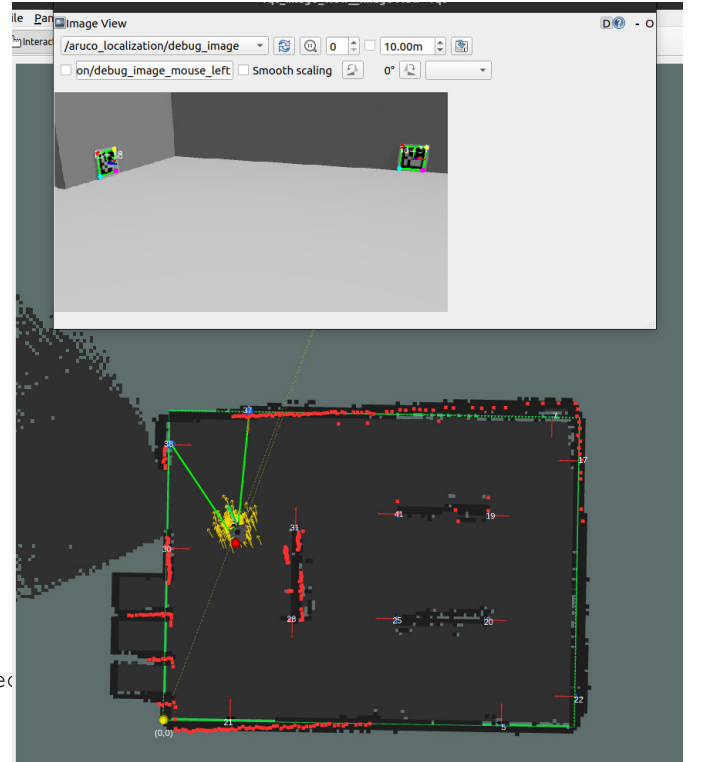


Figure 4. ArUco beacon localization, side by side. Left: the robot in the Gazebo digital twin observing the floor markers. Right: the corresponding RViz 2 view with the detected marker poses and the resulting rescue seed used to re-centre the MCL particle cloud.

a) *Global planner* — A^* : Plans over an inflated costmap (default 0.30 m, hard floor 0.15 m) on an 8-connected grid. Each node expands with cost

$$f(n) = g(n) + h(n), \quad h(n) = \|n - \text{goal}\|_2,$$

where $g(n)$ accumulates diagonal-aware step costs plus the inflation penalty of each traversed cell, and the Euclidean heuristic $h(n)$ keeps the search admissible (it never over-estimates) so the returned path is optimal for the chosen costmap. An inflation **fallback ladder** (0%, 25%, 50%, 75%) retries with reduced margins if the goal cell is occupied after inflation, so the robot always attempts a plan even when a pallet has been parked tight against a wall.

b) *Path follower* — *pure-pursuit PD*: Tracks the planned path at $v = 0.10$ m/s with a 0.50 m look-ahead. Given the look-ahead point at lateral offset y and range L_d in the robot frame, the pursuit curvature is

$$\kappa = \frac{2y}{L_d^2}, \quad \omega = \text{clip}(K_p \kappa v + K_d \dot{e}, \pm \omega_{\max}),$$

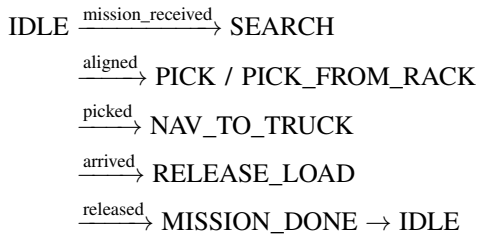
with a low-pass derivative filter ($\tau = 0.2$ s) on the heading error e to damp oscillation. A **reverse maneuver** is triggered when the look-ahead point lies behind the robot and within 0.6 m, allowing recovery from over-shoots instead of a wide loop-around.

c) *Reactive avoidance* — *Bug1 + local costmap*: On obstacle hit the robot switches to Bug1 wall-follow (target distance 0.45 m, right-hand rule). The leave-point is tracked; three watchdogs bound the maneuver: 25 s in-vicinity, 90 s absolute, and 15 s no-progress. The **local costmap** (`local_costmap.py`) provides a 2 m rolling patch with exponential decay ($\tau = 0.7$ s), giving reactive avoidance a temporally-smoothed obstacle signal.

Key tuned parameters: $K_p = 1.5$, $K_d = 0.5$, $\omega_{\max} = 0.21$ rad/s; goal tolerance 0.03 m, heading tolerance 0.04 rad; approach speed floor 0.04 m/s.

C. MR. Mission Control — YASMIN State Machine

The `state_machine_node` hosts a YASMIN finite-state machine with eight states:



Any state can transition to `MISSION_FAILED` on timeout or stop command, which resets back to `IDLE`.

a) *Mission types*: two mission types are supported:

- **Rollers → Truck (Mission 1)**: the robot navigates to the roller staging area, aligns using the E80 logo classifier for a vision stop, picks the pallet, then navigates to the



Figure 5. Navigation in action: the A^* global plan, the pure-pursuit look-ahead target, and a Bug1 boundary-following detour when a previously unmapped obstacle blocks the planned path.

correct truck (identified by logo: amazon/pepsi/walmart) and releases.

- **Racks → Truck (Mission 2)**: same flow but the pick is from a two-level rack (`PICK_FROM_RACK` state); the lifter reaches level 5 for the upper shelf.

b) *Shared blackboard*: States do not call each other directly; they communicate through a single YASMIN blackboard whose keys are written by ROS 2 callbacks and read by the state `execute` methods (Table II). This keeps the FSM logic pure — a state’s outcome is a function of the blackboard snapshot alone — which is what makes the `step` and `force_outcome` debug commands deterministic and safe to use mid mission.

c) *Live debugger*: A `/sm/control` topic accepts JSON commands at runtime: `pause`, `resume`, `step`, `set_step_mode`, `force_outcome`, `abort`. Every transition is published to `/sm/transition`; the full blackboard is snapshotted periodically to `/sm/blackboard`, which the

Table II
PRINCIPAL YASMIN BLACKBOARD KEYS

Key	Meaning / writer
mission	active mission JSON (source, dest, type)
robot_pose	latest /slam_pose sample
aligned	logo/QR alignment converged flag
lifter_done	last /lifter_level ack
stop_flag	protective-stop latch (voice/dashboard)
step_mode	debugger single-step gate

dashboard renders live so an operator can watch the mission advance state by state.

D. M2. FPGA Lifter — SPI HAL

The `lifting_node` translates `/lifter_level` (UInt8, range 0–5) into SPI commands to the Tang Nano 20K FPGA (bus 0, device 0, 1 MHz, mode 0). The FPGA maps each 3-bit binary level to a servo angle:

Table III
LIFTER LEVEL ENCODING (SPI → FPGA → SERVO)

Level	Binary	Servo	Use
0	000	95°	Rest / floor transport
1	001	70°	Low travel height
2	010	65°	Intermediate
3	011	40°	Pick pallet on floor
4	100	25°	Carry / transport
5	101	0°	Pick pallet rack level 2

A pluggable HAL abstraction (`lifting/hal/`) provides JetsonHAL (SPI via `spidev`), MockHAL (simulation), and the legacy JetsonGPIOHAL (3 GPIO pins). The active HAL is selected by `lifting_params.yaml`. Level state is cached to `/tmp` so a mid-mission Jetson restart does not lose the current fork height.

Final demonstration video

Full pick-and-place run: Mission 1 (Rollers → Truck) and Mission 2 (Racks → Truck) on the physical robot. Shows SLAM, ArUco rescue, navigation, logo stop, pick, and delivery.

[Two Missions](#)

Figure 6. Final demonstration: complete autonomous missions on hardware.

E. M2. Hardware and Electronics

The robot is a differential-drive Puzzlebot. Two DC gearmotors are driven through an H-bridge whose direction and PWM lines are commanded by a micro-ROS firmware that subscribes to `/cmd_vel` and closes a low-level velocity loop on the wheel encoders (`/VelocityEncR`,

`/VelocityEncL`). The differential kinematics that the controller inverts to produce wheel set-points are

$$v = \frac{r}{2}(\omega_R + \omega_L), \quad \omega = \frac{r}{b}(\omega_R - \omega_L),$$

for wheel radius r and track width b . Sensing is an RPLidar A1 (`/scan`) and a forward camera (`/cam_img`); the lifter is a servo driven by the Tang Nano 20K FPGA.

A recurring failure mode early on was a **voltage brownout**: a sudden velocity step drew a current spike through the H-bridge that dropped the rail below the Jetson’s reset threshold, rebooting the on-board stack mid mission. The `twist_relay` ramp filter (Section II) bounds $|\Delta v|$ per control tick and was the single change that made long autonomous runs reliable.

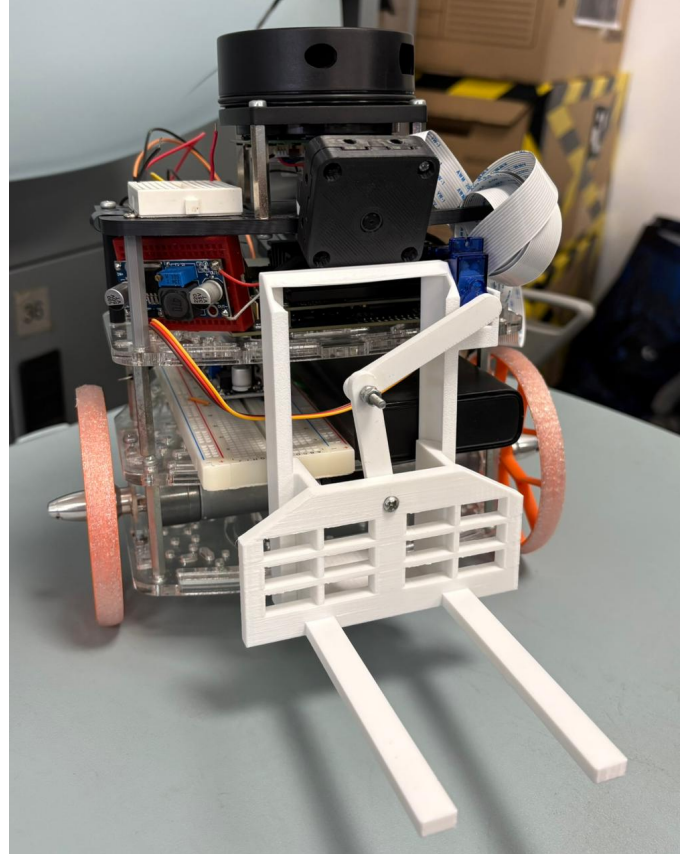


Figure 7. The Puzzlebot AMR platform: Jetson Nano compute, RPLidar A1, forward camera, H-bridge motor driver, and the Tang Nano 20K FPGA servo lifter.

F. MR. Visual Alignment and Perception

a) *E80 logo classifier* (`logo_classifier.py`): A YOLOv8 model (`weights.pt`) runs at 15 Hz, detects the three company logos (amazon, pepsi, walmart) on the trucks and pallets, sorts detections left-to-right by bounding-box centre, and publishes the ordered list as JSON to `/logo_detection`. Confidence threshold 0.35.

During SEARCH the state machine uses the logo stop to halt precisely in front of the pallet; during RELEASE_LOAD it

selects the correct truck by matching the pallet’s QR-decoded company name (with spelling-insensitive aliases: WOLMAR → walmart, EMEZON → amazon, POPSI → pepsi) to the logo observed on each dock.

b) *QR docking* (*qr_quad_alignment.py*): The parabola + wheel-odometry strategy: (1) **PLAN** — at detection, snapshot the QR pose and plan a parabola $y = ax^2$ in the robot frame; reset wheel odometry; (2) **FOLLOW** — pure pursuit on the parabola using only *odometry_tracker.py*, immune to occlusion while the robot’s body blocks the marker; (3) **DOCK** — final straight approach with QR feedback if visible, open-loop otherwise.

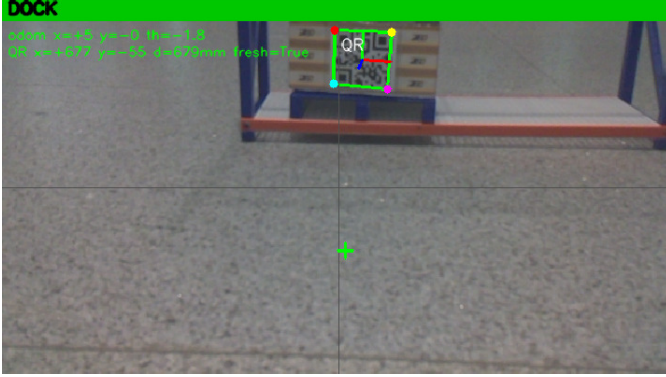


Figure 8. QR docking. The robot snapshots the QR pose, plans a parabola $y = ax^2$ in its own frame, and follows it on wheel odometry — staying aligned even when its chassis occludes the marker during the final approach.

c) *ArUco detection* (*aruco_detector.py*): Compatible with OpenCV 4.6 and 4.7+ via a version-detection shim. Publishes 3-D poses (*rvec* + *tvec* → quaternion) to */aruco_poses* (*PoseArray*) and IDs to */aruco_ids*.



Figure 9. Perception during *RELEASE_LOAD*: the YOLOv8 classifier localizes the three dock logos while the QR decoder reads the pallet’s company tag for truck selection.

G. M4. Voice Control

The *voice_node* implements a fully custom speech-recognition pipeline with no pre-trained models:

$$\text{Audio}_{16\text{ kHz}} \xrightarrow{\text{VAD}} \xrightarrow{\text{MFCC}} \xrightarrow{\text{VQ}_{K=256}} \xrightarrow{\text{HMM}} \text{command}$$

VAD uses an adaptive RMS threshold (0.015); MFCC features are quantised against per-word codebooks (VQ, $K = 256$ centres) and then scored by per-word left-to-right HMMs. The word with the highest log-likelihood wins.

a) *Feature extraction*: Each VAD-gated utterance is framed (25 ms windows, 10 ms hop) and reduced to MFCCs: the magnitude spectrum is mapped through a mel filterbank and the log-energies are decorrelated by a DCT,

$$c_n = \sum_{m=1}^M \log E_m \cos \left[\frac{\pi n}{M} \left(m - \frac{1}{2} \right) \right],$$

keeping the low-order coefficients c_n as the per-frame feature.

b) *Recognition*: Each frame vector is replaced by its nearest VQ codeword, turning the utterance into a discrete symbol sequence $O = o_1 \dots o_T$. A separate left-to-right HMM λ_w is trained per vocabulary word, and the recognized word maximizes the forward likelihood

$$\hat{w} = \arg \max_w P(O | \lambda_w),$$

evaluated with the forward algorithm. Because every word has its own small model and the front-end is plain DSP, the whole recognizer runs in real time on the laptop CPU with no GPU and no pre-trained network — the explicit design goal of avoiding Whisper.

Commands are parsed into three categories:

- **Mission** (start, stop, break, resume, continue, leave) — published to */mission* or the YASMIN blackboard.
- **Control** (lift, lower) — published to */lifter_level*.
- **Motion** (forward, backward, left, right) — published at 0.1 Hz to */cmd_vel_in* during the active window.

H. MR. Web Dashboard

The Flask + SocketIO dashboard (port 8080) provides:

- **Live camera stream** (MJPEG) from */cam_img*.
- **SLAM map canvas** — *ros_bridge.py* encodes each */map* (OccupancyGrid) update as PNG and pushes it via SocketIO; the robot pose is overlaid.
- **Telemetry panel** — pose (*/slam_pose*), YASMIN state (*/robot_state*), active mission, lifter level.
- **Mission control** — dispatch missions (type, source, destination), abort / pause / resume the state machine.
- **Waypoint capture and map save** buttons.
- **Voice-command panel** — browser microphone → WebM → pydub decode → HMM recognizer → */voice_command*.
- **Safety watchdog** — 350 ms dead-man’s switch publishes zero velocity if no teleoperation arrives.

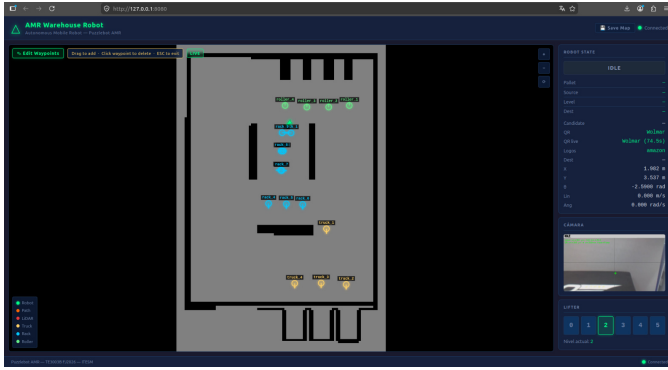


Figure 10. Flask + SocketIO web dashboard (port 8080). Panels (left to right): SLAM occupancy-grid map with robot pose overlay; MJPEG camera feed; telemetry (YASMIN state, pose, lifter level); mission dispatch and control buttons.

I. MR. Digital Twin

The Gazebo digital twin is not a physics simulation; it is a live **environment visualizer** that mirrors the physical robot's estimated pose in real time:

- `gemelo_mirror` subscribes to `/slam_pose` and teleports the virtual robot via Ignition's `set_pose` service at every SLAM update.
- The `almacen_racks.world` scene contains measured-to-reality rack and roller positions (`config/rack_groups.yaml`, `roller_groups.yaml`); the layout is regenerated from `scripts/gen_warehouse.py`.
- Textured ArUco markers match the physical placement so the visualizer can be overlaid on `aruco_map.yaml`.

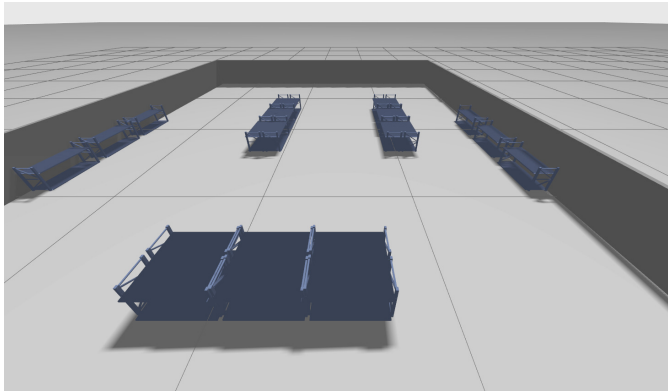


Figure 11. Live Gazebo digital twin. The virtual Puzzlebot tracks the real robot's `/slam_pose` in real time via the `gemelo_mirror` node. Racks, rollers, walls, and textured ArUco markers match the physical track layout.

J. M1. Industrial Standards

a) ISO 10218-1/2 (Industrial robot safety):

- **Speed and separation:** maximum translational velocity capped at 0.10 m/s (`nav_params`); Bug1 maintains a 0.45 m wall-separation distance.

- **Protective stop:** stop voice token and dashboard abort button publish zero velocity and raise `stop_flag` in the YASMIN blackboard within one control cycle (< 100 ms).
- **Anti-brownout relay:** the `twist_relay` ramp prevents abrupt current spikes that could damage the H-bridge.

b) *ISO 13482 (Personal care robots — safety):* The OWL ontology (Sprint 3) was extended with `HazardZone` and `SafetyConstraint` classes linked to the robot's workspace zones. The dead-man's switch in the dashboard (350 ms watchdog) fulfils the unintended-motion requirement for a teleoperated service robot.

IV. CHALLENGES OVERCOME

- **Real-world SLAM smearing:** three independent root causes (DDS multicast collision, clock offset rejecting TF frames, QoS mismatch silently dropping scans) all produced the same symptom. Each was diagnosed and fixed: `IGN_IP=localhost`, wall-clock time with 0.3 s transform tolerance, and matched QoS profiles.
- **MCL drift in low-feature corridors:** the aisle between racks has parallel featureless walls that make the particle filter lose orientation. The ArUco beacon layer resolves this: a single visible marker re-seeds the belief, and the laser immediately snaps the particles to the walls.
- **PnP ambiguity for floor markers:** standard OpenCV `solvePnP` returns two flip solutions for upward-facing markers. `aruco_localization` tests four in-plane conventions and selects the solution with maximum Z-axis uprightness, making the estimate immune to the floor-marker ambiguity.
- **Logo mis-spelling in QR payloads:** physical QR codes encode company names with inconsistent spelling ("Wolmar", "Emezon", "Popsi"). The `logo_aliases` map in `zones.yaml` normalises all variants case- and punctuation-insensitively before truck selection.
- **Vision occlusion during docking:** the robot's chassis blocks the QR marker at close range. The parabola + odometry strategy (`qr_quad_alignment.py`) eliminates the dependency on continuous vision during the FOLLOW phase.
- **Voltage brownout on acceleration:** rapid velocity commands caused H-bridge current spikes that reset the Jetson. The `twist_relay` ramp filter limits Δv per tick.
- **YASMIN + ROS 2 thread safety:** the state machine runs in a background thread while ROS 2 callbacks update the blackboard from the executor thread. CPython's GIL guarantees atomicity for the scalar and dict types used on the blackboard for this use case.

V. RESULTS

Both target missions were executed successfully end-to-end on the physical robot:

- **Mission 1 — Rollers → Truck:** robot navigates to the roller staging area, uses the E80 logo classifier for a vision stop, picks the pallet at floor level (lifter level 3), navigates to the

correct truck identified by logo matching, and deposits the pallet.

- **Mission 2 — Racks → Truck:** same pipeline with the `PICK_FROM_RACK` state; the lifter reaches level 5 (0°) to engage pallets on the upper shelf.

Table IV summarizes the operating characteristics observed during the validation sessions. The headline result is qualitative — **both missions complete autonomously, repeatably** — but the quantitative figures below capture the tuning targets the stack was brought to. Figure 12 shows a representative run.

Table IV
OBSERVED SYSTEM CHARACTERISTICS

Quantity	Value
MCL / SLAM update rate	10 Hz
Particle count	600
Logo classifier rate	15 Hz
Nominal cruise speed	0.10 m/s
Goal position tolerance	0.03 m
Goal heading tolerance	0.04 rad
Bug1 wall-separation distance	0.45 m
Protective-stop latency	< 100 ms
Missions validated on hardware	2 / 2

— C++ MCL SLAM with CUDA scoring and graph-SLAM loop closure, ArUco beacon localization as an MCL rescue layer, A* + Bug1 + pure-pursuit navigation, a YASMIN orchestrator, YOLOv8 logo-based truck selection, QR docking via parabola + odometry, an MFCC + VQ + HMM voice pipeline, a live digital twin, and a Tang Nano 20K SPI lifter — demonstrates that the entire AMR autonomy stack can be implemented from first principles without relying on Nav2, Whisper, or `robot_localization`. All 23 planned tasks were completed before the final demonstration. The modular package structure and distributed Jetson + laptop deployment proved robust throughout integration testing and the final demonstration.

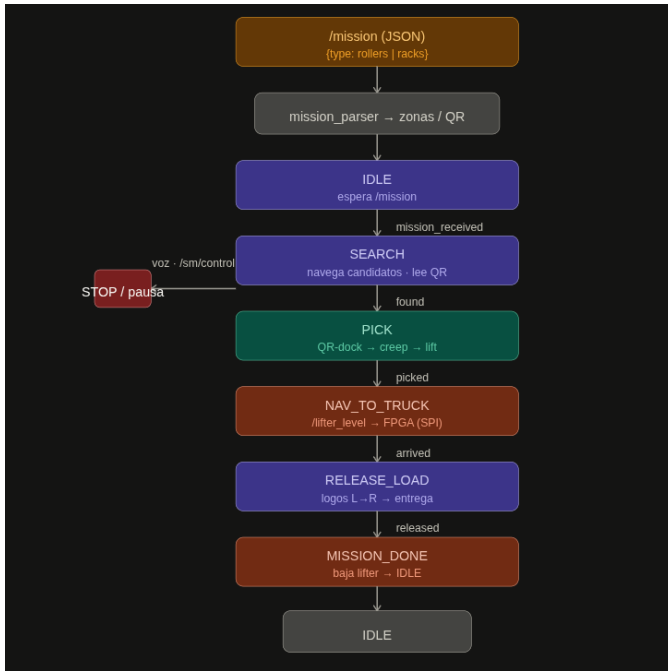


Figure 12. Representative Mission 1 sequence on hardware: navigation to the rollers, logo-based vision stop, pallet pick, transport, truck alignment, and release.

A demonstration video of both missions is linked in Figure 6.

VI. CONCLUSION

The Puzzlebot AMR completed both target missions autonomously on the physical hardware. The fully custom stack